

Parallel Project Report: Distributed Sparse Matrix-Vector Multiplication (SpMV) with MPI

Francesco Zadra (ID: 244544)

University of Trento

Department of Information Engineering and Computer Science (DISI)

Email: francesco.zadra-1@studenti.unitn.it

Course: Introduction to Parallel Computing (2025-2026)

Github repository: <https://github.com/FranZadra/PARCO-Computing-2026-244544>

Abstract—Sparse matrix-vector multiplication (SpMV) is an important computational kernel in high-performance computing (HPC), widely used in scientific applications. However, its performance on modern processors is often limited due to its irregular memory access patterns and its memory-bounded nature. This paper presents a reproducible study that evaluates and compares a distributed-memory CSR-based SpMV implementation on a multicore platform. On an Intel Xeon Gold 6252N, we perform repeated tests with different MPI configurations, up to 128 total processes, across real-world and synthetic matrices to compare strong/weak scaling, speedup and execution time. Experimental results show strong scalability up to 32–64 processes; beyond that, communication overhead outweighs computational gains, limiting parallel efficiency. Additionally, we evaluated a hybrid MPI+OpenMP implementation. Results indicate that while the hybrid approach improves performance up to 4 processes, the combination of OpenMP scheduling overhead and MPI communication reduces efficiency at higher scales.

Source code, scripts and instructions for full reproducibility are provided. (see Sec. VII)

Index Terms—Parallel computing, MPI, SpMV, CSR format, Performance analysis

I. INTRODUCTION

Problem. Sparse Matrix-Vector Multiplication (SpMV) is one of the most performance-critical kernels used in scientific and engineering computations, as it is a central part in many iterative solving algorithms. Common usages of SpMV include big data analytics, machine learning model training, GPU computing, graph processing and numerical methods for solving large sparse linear systems. [1]–[3]

SpMV has poor performance on modern processors as it is strongly memory-bounded and characterized by irregular memory access patterns, which make it difficult to optimize efficiently [4].

This paper aims to provide a clear evaluation of CSR-based SpMV performance on modern multicore architectures and to outline the main limitations that arise when scaling parallel execution in Distributed-Memory systems.

Objectives. This research focuses on the following goals:

- (i) the development of a parallel CSR SpMV implementation for distributed-memory systems using the Message Passing Interface (MPI);
- (ii) an experimental evaluation using both synthetic and real representative matrices to analyze weak and strong scaling,

execution time, speedup, and performance bottlenecks;

(iii) a comparison between pure MPI and a hybrid MPI + OpenMP implementation;

(iv) the documentation of a reproducible workflow.

Contributions. This research provides a reproducible study that evaluates a CSR SpMV multiplication using both synthetic and real representative sparse matrices and adopts appropriate performance metrics and visualizations to analyze performance differences and highlight inherent limitations in modern systems. The paper concludes with suggestions for more in-depth analysis and future research.

II. STATE OF THE ART

The core Sparse Matrix-Vector Multiplication (SpMV) operation is formally defined as

$$\mathbf{y} \leftarrow \mathbf{y} + \mathbf{A}\mathbf{x}, \quad (1)$$

where $\mathbf{A}$ is a sparse matrix and $\mathbf{x}, \mathbf{y}$ are dense vectors. We refers to $\mathbf{x}$ as the *source vector*, while $\mathbf{y}$ is the *destination vector*.

Although conceptually simple, SpMV is a performance-critical kernel in a wide range of High-Performance Computing (HPC), scientific, and engineering applications [3]. This is mainly due to its low arithmetic intensity and irregular memory access patterns. As highlighted in the study by Williams and Oliker [6], memory bandwidth can become a significant bottleneck in chip multiprocessor (CMP) architectures, affecting SpMV performance.

The most common and general-purpose data structure used to store a sparse matrix for SpMV is the Compressed Sparse Row (CSR) format [5], due to its compact representation and its row-oriented traversal compatibility with modern CPU architectures. Researchers have explored optimized extensions of CSR, such as CSR5 [9] and Block-CSR (BCSR) [8], which aim to improve load balance and vectorization in specific matrix classes. However, this study focuses exclusively on standard CSR.

Although many recent studies focus on SpMV performance on GPUs or heterogeneous systems through frameworks such as CUDA, OpenACC, or SYCL [7], [10]–[12], this paper concentrates specifically on modern multicore CPUs using MPI,

which is the most widely adopted standard for parallelization in distributed-memory systems.

To summarize, this work aims to provide a pragmatic and reproducible approach to test and analyze the performance of a CSR-based SpMV implementation on modern multicore CPU architectures. It evaluates distributed-memory execution using MPI, comparing also pure MPI against hybrid MPI+OpenMP configurations across multiple matrices. Particular attention is given to strong and weak scaling, speedup, inter-process communication overhead, and the practical parallelization limits inherent in distributed computing.

III. METHODOLOGY

A. Data structures

- **Sparse matrix A :** stored in *Compressed Sparse Row (CSR)* format [5], composed of:
 - `row_ptr`: array of size `n_rows + 1` storing the index boundaries of each row,
 - `col_ind`: array storing the column index of each non-zero element,
 - `vals`: array storing the non-zero values of the matrix.
- **Dense input vector $\mathbf{rvec}$:** randomly generated, with length equal to the number of columns of the matrix.
- **Dense result vector $\mathbf{res}$:** with length equal to the number of rows of the matrix, used to store the output of the SpMV operation.

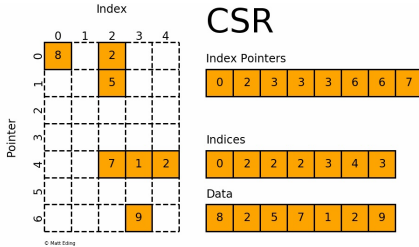


Fig. 1. Example of CSR. [5]

B. SpMV Algorithm overview

Algorithm 1: Distributed SpMV

Input: CSR local matrix (`vals`, `col_ind`, `row_ptr`), random vector (`localRandVec`)

Output: vector `localResult` = $A \cdot \mathbf{r}$

The code computes the product of a portion of a sparse matrix and a vector by iterating through each matrix row, summing the contributions of its non-zero elements. So, for each non-zero element of the sparse matrix, the algorithm computes a product and accumulates the sum (2 FLOPS, i.e. two floating point operations: one multiplication and one addition). Depending on the column index, the required vector element may be stored locally by the rank or represent a "ghost" value that must be exchanged between ranks via MPI

communication before the local SpMV computation begins. Pseudocode:

```
for i = 0 to local_rows-1:
    sum = 0.0
    for j = row_ptr[i] to row_ptr[i+1]-1:
        global_col = col_ind[j]
        col_owner = global_col % comm_size

        if col_owner == rank:
            local_idx = global_col / comm_size
            vect_val = localRandVec[local_idx]
        else:
            ghost_idx = ghost_to_local[global_col]
            vect_val = ghost_values[ghost_idx]

    sum += vals[j] * vect_val

localResult[i] = sum
```

C. Parallelization (MPI)

To exploit parallelism, we use MPI, a standard API for distributed-memory programming. MPI enables a program to run across multiple processes that may be on different nodes, each with its own memory space.

In this implementation, the process with rank 0 distributes a subset of the matrix and the corresponding elements of the vector to all other ranks, using 1D cyclic partitioning (see row ownership rule below). Communication occurs to exchange "ghost" values, which are elements of the vector needed by one process but owned by another.

MPI was chosen as the parallel programming model because it allows performance scalability across distributed systems. Different communication patterns and data distributions can be tested to evaluate how communication overhead, computation balance, and network latency affect the global performance of the SpMV multiplication.

Row ownership rule: $\text{owner}(i) = i \bmod P$ where P is the number of processes and i is the row index.

IV. EXPERIMENTS AND SYSTEM DESCRIPTION

A. Platform

- **Architecture:** x86_64, Little Endian
- **CPU model:** Intel Xeon Gold 6252N @ 2.30GHz
- **Total CPUs:** 144 physical cores distributed across 6 sockets (24 cores per socket)
- **Total nodes:** 6
- **Threads per core:** 1 (for pure MPI tests)
- **Cache hierarchy:**
 - L1 cache: 32 KB per core
 - L2 cache: 1 MB per core
 - L3 cache: 36.6 MB per core

SpM	Name	Dim.	NNZ
	bcsstk27	1224×1224	28.7K
	Andrews	60K×60K	410K
	tmt_sym	726K×726K	2.9M
	msdoor	415K×415K	10.3M
	Hook_1498	1.5M×1.5M	31.2M

TABLE I. Real sparse matrices used in the benchmark.

B. Environment

The tests have been executed on multiple *HPC* nodes of the UNITN University cluster. The operating system is Linux, and job submission is handled through the PBS queue management system.

Detailed Job resources requested:

- **Queue type:** public *short_cpuQ* (6h wall-time)
- **Number of nodes:** 6
- **CPUs per node:** 24
- **RAM memory per node:** 16 GB
- **Total CPUs:** 144

Experiments were compiled using *mpicc* for *MPI* support and optimized with *-O3*. Hybrid *MPI+OpenMP* experiments were compiled adding the *-fopenmp* flag.

C. Experimental setup

1) *Dataset:* To evaluate and test the different *MPI* setups, we benchmarked the implementation using five sparse matrices from the SuiteSparse collection [13] for strong scaling analysis, and we used synthetic matrices created randomly for weak scaling analysis.

All SuiteSparse matrices are stored in *.mtx* files in *COO* format and are pattern symmetric, numerically symmetric, positive definite.

The code automatically reads the matrices in *COO* format and converts them to *CSR*, prior to processing.

For weak scaling analysis, we generated synthetic matrices with a fixed workload per process: 10,000 rows per rank and 50 non-zero elements per row.

2) *Experimental procedure and baseline:* The baseline is an *MPI* implementation of the Sparse Matrix-Vector multiplication (see Sec. III for the code). Rank 0 reads and distributes the matrix to all other ranks using 1D cyclic partitioning. Each rank then computes its local SpMV multiplication, exchanging ghost values via *MPI* collective *MPI_Alltoallv*. A Bash script, executed by the PBS job, runs all *MPI* parallel configurations. The tested parameters includes:

- **Number of processes:** 1, 2, 4, 8, 16, 32, 64, 128

Hybrid *MPI+OpenMP* tests are run with a *dynamic* schedule type (with chunk size 64). The number of *OpenMP* threads per *MPI* process is dynamically adjusted based on the

available cores per node (up to 24 threads).

Each configuration is tested 10 times after an initial warm-up to limit cold-cache effects.

3) *Metrics collected:* The results are saved in *CSV* files that give us the following metrics:

- Execution time [ms]
- Communication time [ms]
- Number of local non-zero elements
- Number of "ghost" entries
- Local floating point operations [FLOPS]

D. Data analysis

After getting all the data, a *Python* script analyzes all the collected data and generates visualizations for the following comparisons and models:

- **Strong scaling** trends: the scalability of a program should be measured by increasing the resources and keeping the problem size (Amdahl [14] [15])
- **Computation vs Communication Time** comparisons (with different numbers of processes)
- **Weak scaling** trends: the scalability of a program should be measured by increasing the resources and the problem size (Gustafson [14] [16])
- **MPI+OpenMP** comparisons

V. RESULTS & DISCUSSION

This section provides only some of the visualizations, the full set of plots for tested matrices is available in the GitHub repository referenced in the paper. (see. Sec VII)

A. Strong scaling speedup

We tested our *MPI* SpMV implementation using up to 128 processes on real matrices. The results exhibit strong scalability in our data, with parallelization yielding consistent performance improvements for this kernel [17] [18]. In fact, the experimental results follow the same speedup pattern as the theoretical curve computed using Amdahl's Law [15]. However, at 128 processes, the speedup degrades in most cases, particularly for small matrices, where communication overhead exceeds the benefits of increased computation. Fig. 2 shows strong scaling speedup for *Andrews* matrix.

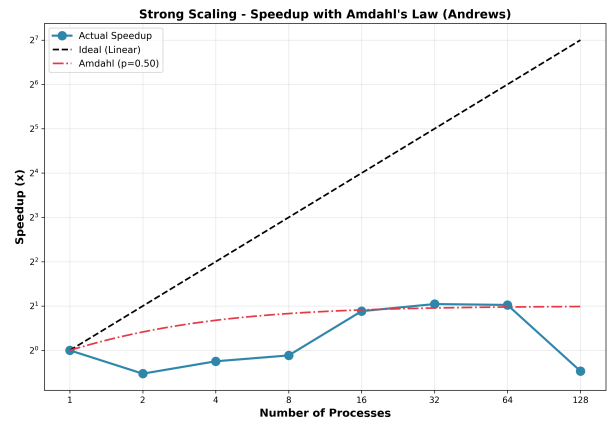


Fig. 2. Andrews: strong scaling speedup

B. Computation vs Communication time

Experimental results provide precise measurements of computation and communication times. Computation times improve significantly with parallelization, demonstrating strong scalability [19]. However, communication times increase with the number of processes, causing total execution time to decrease up to 32–64 processes. Beyond this point, communication overhead dominates, degrading performance, except for matrix *tmt_sym*, where benefits persist up to 128 processes. Fig. 3 shows *Andrews* matrix computation and communication times.

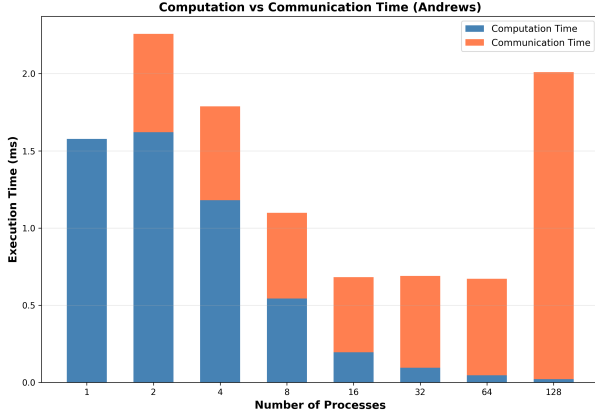


Fig. 3. Andrews: computation vs communication time

C. Weak scaling

We also tested weak scaling using synthetic matrices, where the problem size scales with the number of processes [18]. Computation times remain nearly constant, while communication times grow significantly beyond 16 processes, indicating that communication overhead limits scalability and prevents ideal linear weak scaling predicted by Gustafson's Law [16]. Fig. 4 reports the execution times obtained with synthetic matrices.

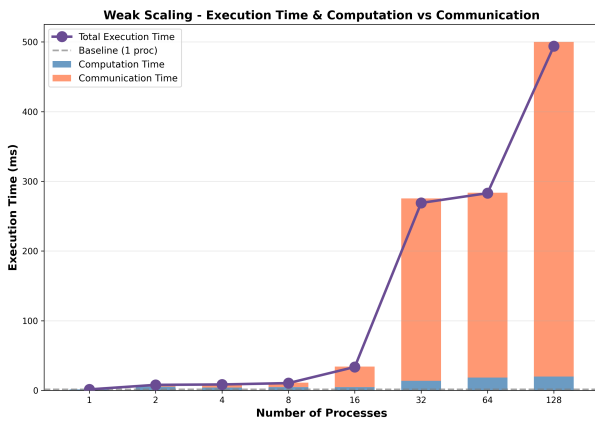


Fig. 4. weak scaling: execution time and computation vs communication trend

D. MPI+OpenMP

We developed an MPI+X implementation incorporating OpenMP parallelization to assess whether intra-node multi-

threading enhances performance. We configured dynamic scheduling with 64-element chunks and replicated the tests from the pure-MPI version. Results indicate that while the hybrid approach improves performance up to 4 processes, the combination of OpenMP scheduling overhead and MPI communication reduces efficiency at higher scales [20]. Fig. 5 shows MPI+OpenMP strong scaling speedup compared to MPI-only speedup.

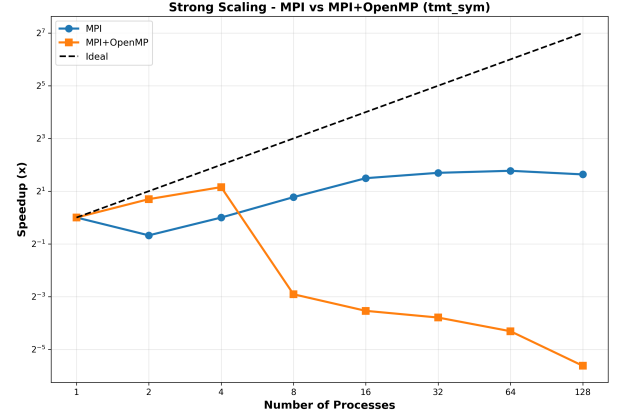


Fig. 5. tmt_sym: MPI+OpenMP speedup

VI. CONCLUSIONS & FUTURE WORK

In this research, we implemented and evaluated a distributed-memory CSR-based SpMV using MPI. We demonstrated that parallelization yields strong scaling and improved performance especially on large matrices such as *msdoor* (10M) and *Hook_1498* (31M non-zero elements). We observed increasing efficiency up to 32–64 processes. Beyond that, communication overhead significantly outweighs the computational gain, reducing parallel efficiency. Future work should explore advanced storage formats (e.g., CSR5, BCSR), alternative data partitioning (e.g., 2D) and optimized I/O approaches (e.g., parallel reading, MPI-IO), benchmarking them against our MPI implementation. Other areas of interest in recent studies are PIM (Processing-in-Memory) architectures and Machine Learning techniques for auto-tuning optimal configurations.

VII. REPRODUCIBILITY & ARTIFACT AVAILABILITY

Git public repository: <https://github.com/FranZadra/PARCO-Computing-2026-244544>

How to replicate: All the necessary information to replicate this study is provided in the `README.md` file in the Git repository, inside the subfolder named "D2".

REFERENCES

- [1] Daniele Buono, Fabrizio Petrini, Fabio Checconi, Xing Liu, Xinyu Que, Chris Long, and Tai-Ching Tuan: "Optimizing Sparse Matrix-Vector Multiplication for Large-Scale Data Analytics." In Proceedings of the 2016 International Conference on Supercomputing (2016)
- [2] AlAhmadi, S.; Mohammed, T.; Albeshri, A.; Katib, I.; Mehmood, R.: "Performance Analysis of Sparse Matrix-Vector Multiplication (SpMV) on Graphics Processing Units (GPUs)." Electronics (2020)
- [3] Gao, J., Liu, B., Ji, W., Huang, H.: "A systematic literature survey of sparse matrix-vector multiplication." arXiv preprint arXiv:2404.06047 (2024).
- [4] Y. Che, C. Xu, J. Fang, Y. Wang and Z. Wang: "Realistic Performance Characterization of CFD Applications on Intel Many Integrated Core Architecture," in The Computer Journal, vol. 58, no. 12, pp. 3279-3294, (Dec. 2015)
- [5] Matt Eding: "Sparse Matrices": <https://matteding.github.io/2019/04/25/sparse-matrices/>
- [6] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel: "Optimization of sparse matrix-vector multiplication on emerging multicore platforms", 2007 ACM/IEEE conference on Supercomputing (SC '07)
- [7] N. Bell, M. Garland: "Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors", NVIDIA
- [8] S. Liu, Y. Zhang, X. Sun and R. Qiu: "Performance Evaluation of Multithreaded Sparse Matrix-Vector Multiplication Using OpenMP," 2009 11th IEEE International Conference on High Performance Computing and Communications, Seoul, Korea (2009)
- [9] Weifeng Liu and Brian Vinter. 2015. CSR5: "An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication."
- [10] P. A. Lane, J. D. Booth: "Heterogeneous sparse matrix-vector multiplication via compressed sparse row format", Parallel Computing, Volume 115 (2023)
- [11] Jianhua Gao, Weixing Ji, and Yizhuo Wang: "Optimization of Large-Scale Sparse Matrix-Vector Multiplication on Multi-GPU Systems." (2024)
- [12] Kinga Stec, Przemysław Stpicyński: "Performance portability of sparse matrix-vector multiplication implemented using OpenMP, OpenACC and SYCL" Future Generation Computer Systems, (2025)
- [13] Timothy Davis: "The SuiteSparse Matrix Collection." <http://sparse.tamu.edu/>
- [14] Flavio Vella, Brahmaiah Gandham: "Lecture 5, Performance model and benchmarking", Introduction to Parallel Computing (2025)
- [15] M. D. Hill and M. R. Marty, "Amdahl's Law in the Multicore Era" in Computer, vol. 41, no. 7, pp. 33-38, (July 2008).
- [16] John L. Gustafson: "Reevaluating Amdahl's law", Commun. ACM 31, 5 (May 1988)
- [17] B. A. Page and P. M. Kogge: "Scalability of Hybrid SpMV on Intel Xeon Phi Knights Landing," 2019 International Conference on High Performance Computing & Simulation (HPCS), Dublin, Ireland, (2019)
- [18] Bahmani, A., & Mueller, F.: "Scalable Performance Analysis of ExaScale MPI Programs through Signature-Based Clustering Algorithms", 28th ACM International Conference on Supercomputing (ICS '14)
- [19] Chen, Wenguang & Zhai, Jidong & Zhang, Jin & Zheng, Weimin: "Log-GPO: An accurate communication model for performance prediction of MPI programs" (2009)
- [20] M. J. Chorley, D. W. Walker: "Performance analysis of a hybrid MPI/OpenMP application on multi-core clusters", Journal of Computational Science, Volume 1, Issue 3 (2010)